

Riassunto Dettagliato e Aggiornato:

Development of an Orchard Inspection Robot: A ROS-Based LiDAR-SLAM System with Hybrid A-DWA Navigation*

Sensors 25 (2025) 6662

NECKER

Basato sul PDF originale + qualche precisazione

9 Dicembre 2025

Sommario

Questo documento presenta un'analisi dettagliata dello sviluppo di un robot mobile per l'ispezione autonoma nei frutteti. Viene descritto un sistema di navigazione SLAM (Simultaneous Localization and Mapping) che utilizza la tecnologia LiDAR e una fusione multi-sensore (LiDAR + IMU). Viene presentato il modello cinematico del robot e le trasformazioni di coordinate basate su URDF all'interno di ROS. Il cuore della navigazione è una strategia di pianificazione del percorso ibrida che combina un algoritmo A^* migliorato (con pesi adattivi) per la pianificazione globale e l'approccio Dynamic Window Approach (DWA) per l'evitamento degli ostacoli in tempo reale. I risultati sperimentali dimostrano l'efficacia del sistema in termini di precisione di arresto ai waypoint ed efficienza nell'evitamento degli ostacoli in ambienti non strutturati.

Indice

1	Introduzione	2
2	Sistema Robotico	2
3	Modello Cinematico e Trasformazioni	3
3.1	Trasformazione del Sistema di Coordinate	3
3.2	Cinematica Diretta e Inversa	3
4	Architettura e Flusso Dati	4
4.1	Configurazione delle Costmap (Mappe di Costo)	4
5	Strategia di Pianificazione del Percorso (Hybrid A^*-DWA)	6
5.1	Algoritmo A^* Migliorato	6
5.2	Algoritmo DWA (Local Planner)	7
5.3	Metodo Ibrido e Integrazione	8
6	Risultati della Simulazione	9
6.1	Confronto Pianificazione Globale	9
6.2	Confronto Pianificazione Locale	9
7	Sperimentazione sul Campo	10
7.1	Test di Arresto ai Waypoint	10
7.2	Test di Evitamento Ostacoli	10

1 Introduzione

L'automazione del monitoraggio agricolo è fondamentale per migliorare l'efficienza della gestione dei frutteti e ridurre l'intensità del lavoro. I robot mobili autonomi offrono una soluzione promettente rispetto alla sorveglianza con droni o postazioni fisse, permettendo una raccolta dati dettagliata a livello del suolo. Tuttavia, la navigazione in ambienti non strutturati (come i frutteti) presenta sfide significative, in particolare quando i segnali GNSS sono inaffidabili a causa dell'ombreggiamento della chioma degli alberi.

Questo studio propone un sistema basato su LiDAR SLAM, preferito per la sua elevata precisione, robustezza in tempo reale e adattabilità alle condizioni di luce. Le principali innovazioni discusse includono:

- Sviluppo di un robot mobile con sensori multi-modali strettamente accoppiati (LiDAR e IMU).
- Modellazione cinematica e trasformazioni di coordinate tramite URDF per la visualizzazione in ROS.
- Una strategia di pianificazione ibrida che unisce un A^* migliorato (per l'efficienza globale) e DWA (per la reattività locale).

2 Sistema Robotico

Il robot è costituito da una piattaforma mobile a quattro ruote motrici indipendenti con sterzo differenziale. Le componenti principali includono:

- **Chassis:** Dotato di sospensioni indipendenti a doppio braccio per minimizzare l'impatto sugli attuatori.
- **Unità di Calcolo:** Computer industriale (NP-6122-JH3) che elabora i dati e invia comandi al telaio.
- **Sensori:**
 - IMU (WT901C-9) per misurare velocità, accelerazione e assetto.
 - LiDAR (RS-LiDAR-16) per la raccolta dati ambientali e costruzione mappe.
 - Modulo GNSS e telecamera gimbal su asta elevabile per l'acquisizione immagini.

3 Modello Cinematico e Trasformazioni

Il robot è descritto tramite il formato URDF (Unified Robotics Description Format), un XML usato in ROS per descrivere le caratteristiche fisiche del robot. Per gestire la complessa struttura, il modello 3D progettato in SolidWorks è stato convertito in un file URDF compatibile con ROS.

3.1 Trasformazione del Sistema di Coordinate

La trasformazione tra due sistemi di coordinate cartesiane segue il modello di trasformazione a sette parametri di Helmert. La relazione tra il sistema di coordinate del veicolo (C) e quello del LiDAR (S) è fondamentale. Data una rotazione δ, φ, θ rispettivamente attorno agli assi x, y, z, e assumendo un fattore di scala $k = 1$, la trasformazione completa è descritta dalla seguente equazione:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ z_t \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \delta & -\sin \delta \\ 0 & \sin \delta & \cos \delta \end{bmatrix} \begin{bmatrix} \cos \varphi & 0 & \sin \varphi \\ 0 & 1 & 0 \\ -\sin \varphi & 0 & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ z_s \end{bmatrix} \quad (1)$$

Questa trasformazione permette di fondere i dati della nuvola di punti LiDAR con il sistema di riferimento del veicolo.

3.2 Cinematica Diretta e Inversa

Il robot utilizza un modello a trazione differenziale su quattro ruote. Definiti V_L e V_R come le velocità delle ruote sinistra e destra, v_{cy} come la velocità lineare del telaio e ω_c come la velocità angolare, le relazioni cinematiche sono:

Cinematica Diretta (dalle ruote al telaio):

prende in input la velocità delle ruote (giri per minuto per esempio) e restituisce la velocità del telaio.

$$\begin{bmatrix} v_{cy} \\ \omega_c \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{c} & \frac{1}{c} \end{bmatrix} \begin{bmatrix} V_L \\ V_R \end{bmatrix} \quad (2)$$

Dove c è la distanza tra le linee centrali delle ruote sinistra e destra.

Cinematica Inversa (dal telaio alle ruote):

prende la velocità del telaio desiderata e calcola quanto devono muoversi le ruote

$$\begin{bmatrix} V_L \\ V_R \end{bmatrix} = \begin{bmatrix} 1 & -\frac{c}{2} \\ 1 & \frac{c}{2} \end{bmatrix} \begin{bmatrix} v_{cy} \\ \omega_c \end{bmatrix} \quad (3)$$

4 Architettura e Flusso Dati

Il nodo `move_base` opera attraverso un flusso dati strutturato all'interno del grafo computazionale di ROS:

- **Input Sensoriali e di Stato:** Il sistema sottoscrive i messaggi LiDAR nei formati `sensor_msgs/LaserScan` e `sensor_msgs/PointCloud` per la percezione degli ostacoli. Sottoscrive inoltre i dati odometrici tramite `nav_msgs/Odometry` e le trasformazioni di coordinate tramite il sistema `tf` per determinare la posizione del robot.
- **Output di Controllo:** Dopo aver elaborato i percorsi ottimali, il nodo pubblica comandi di velocità nel formato `geometry_msgs/Twist` sul topic `cmd_vel`, regolando la velocità lineare e angolare del telaio in tempo reale.

4.1 Configurazione delle Costmap (Mappe di Costo)

Per rappresentare le informazioni sugli ostacoli, il sistema mantiene due costmap distinte: una `global_costmap` per la pianificazione a lungo termine e una `local_costmap` per l'evitamento ostacoli reattivo. La loro configurazione è gerarchica e divisa in tre livelli:

1. Configurazione Comune (General Plugin)

Questo livello definisce i parametri fisici e sensoriali condivisi da entrambe le mappe:

- **footprint:** Definisce l'area fisica occupata dal robot, essenziale per calcolare le collisioni e garantire che il robot passi attraverso spazi stretti.
- **observation_sources:** Impostato su `scan`, specifica che la fonte primaria per l'aggiornamento della mappa è la nuvola di punti LiDAR.
- **obstacle_range (2.5 m):** Definisce la distanza massima entro la quale il sensore inserisce nuovi ostacoli nella costmap. Gli ostacoli rilevati oltre questa distanza vengono ignorati per mantenere la mappa pulita da rumore lontano.
- **raytrace_range (3.0 m):** Definisce la distanza massima per l'operazione di "raytracing" (pulizia dello spazio). Se il sensore scansiona un'area fino a 3 metri e non trova nulla, il sistema rimuove eventuali ostacoli precedentemente segnati in quello spazio. È impostato a un valore superiore all'`obstacle_range` per garantire una gestione coerente dello spazio libero.
- **min_obstacle_height (0.25 m) e max_obstacle_height (0.35 m):** Questi filtri di altezza sono critici per l'operatività nei frutteti. Permettono di filtrare le irregolarità del terreno e l'erba bassa (sotto i 25 cm) e di concentrare l'evitamento su tronchi e ostacoli rilevanti che potrebbero colpire il telaio, evitando falsi positivi.
- **inflation_radius:** Il raggio di espansione attorno agli ostacoli che crea una zona cuscinetto di sicurezza. Questo parametro è stato oggetto di ottimizzazione sperimentale (risultato ottimale a 0.4 m).

2. Configurazione della Global Costmap

Utilizzata dal pianificatore globale (A^*), questa mappa copre l'intero ambiente operativo:

- **global_frame:** Impostato su `map`, riferito al sistema di coordinate fisso del mondo.
- **robot_base_frame:** Impostato su `base_link`, riferito al centro geometrico del robot.
- **update_frequency: 1 Hz.** Poiché la mappa globale contiene principalmente ostacoli statici, una frequenza di aggiornamento bassa è sufficiente e risparmia risorse computazionali.

3. Configurazione della Local Costmap

Utilizzata dal pianificatore locale (DWA), questa mappa è una "finestra mobile" (rolling window) centrata sul robot:

- **Dimensioni e Risoluzione:** Configurate come una finestra di **10 m** $\times$ **10 m** con una risoluzione di **0.5 m**. Questa risoluzione relativamente bassa riduce il carico computazionale, permettendo al DWA di calcolare traiettorie complesse rapidamente.
- **publish_frequency: 2 Hz.** La mappa locale viene aggiornata e pubblicata più frequentemente per rispondere ai cambiamenti dinamici immediati nell'ambiente circostante.
- I frame di riferimento sono allineati alla configurazione globale per garantire la coerenza tra i due pianificatori.

5 Strategia di Pianificazione del Percorso (Hybrid A^* -DWA)

Per garantire una navigazione autonoma efficiente e sicura in un ambiente non strutturato come un frutteto, è stata adottata una strategia ibrida. Questa combina un algoritmo globale A^* migliorato, per definire la traiettoria ottimale a lungo raggio, con l'approccio Dynamic Window Approach (DWA) per la gestione locale della cinematica e l'evitamento degli ostacoli imprevisti.

5.1 Algoritmo A^* Migliorato

L'algoritmo A^* standard valuta i nodi utilizzando la funzione di costo $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo effettivo dal nodo di partenza e $h(n)$ è il costo euristico stimato verso l'obiettivo. In questo studio, la metrica della distanza Euclidea è utilizzata per calcolare entrambi i valori. Tuttavia, l'approccio standard assegna pesi uguali a $g(n)$ e $h(n)$, il che può portare all'espansione di un numero eccessivo di nodi candidati, riducendo l'efficienza di ricerca.

Per ottimizzare questo processo, è stata introdotta una funzione di peso adattiva $w(n)$ applicata alla componente euristica. L'obiettivo è favorire un'ampia esplorazione nelle fasi iniziali (comportamento simile a Dijkstra) e accelerare la convergenza verso l'obiettivo nelle fasi finali (comportamento simile a Greedy Best-First Search). La funzione di costo modificata è definita come:

$$f(n) = g(n) + \left\{ \sin^2 \left[\frac{\pi}{2} \left(1 - \frac{d}{D} \right) \right] + \frac{1}{2} \right\} h(n) \quad (4)$$

Dove:

- d rappresenta la distanza Euclidea dal nodo di partenza al nodo corrente.
- D rappresenta la distanza totale stimata dal nodo di partenza al nodo obiettivo.

Il termine tra parentesi graffe rappresenta il peso dinamico. Questo peso varia in modo sinusoidale, garantendo una transizione fluida all'interno dell'intervallo $[0.5, 1.5]$. I limiti sono stati determinati empiricamente: un peso inferiore a 0.5 risulterebbe in un'esplorazione eccessiva, mentre un peso superiore a 1.5 rischierebbe di compromettere l'ottimalità del percorso a favore della velocità di calcolo. La funzione sinusoidale assicura che il peso sia vicino a 0.5 quando $\frac{d}{D}$ è piccolo (inizio del percorso) e tenda a 1.5 quando $\frac{d}{D}$ si avvicina a 1 (vicino all'obiettivo).

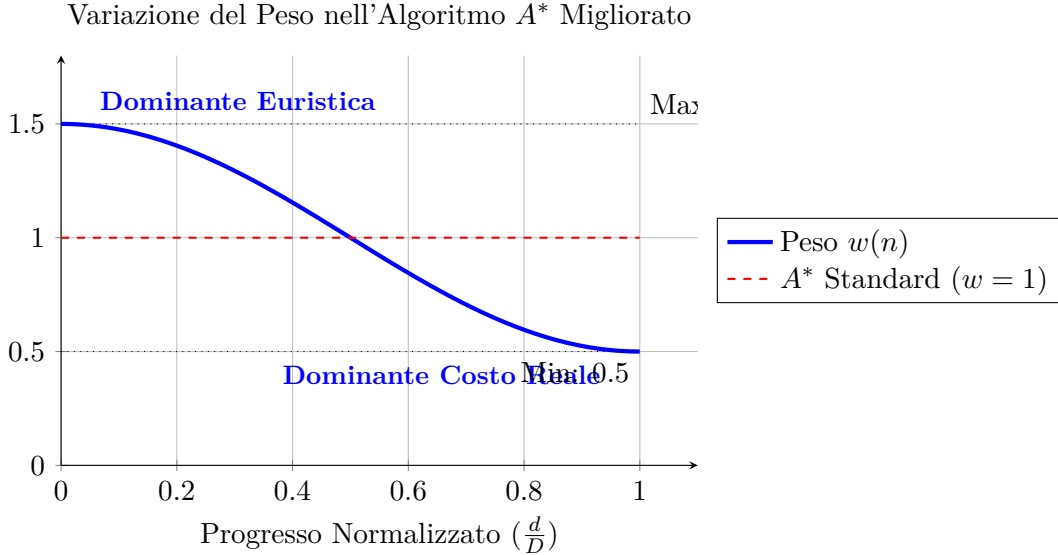


Figura 1: Andamento della funzione di peso adattivo. Il peso decresce sinusoidalmente da 1.5 a 0.5 man mano che il robot si avvicina all'obiettivo ($d \rightarrow D$), passando da un approccio greedy a uno più accurato.

5.2 Algoritmo DWA (Local Planner)

Mentre l' A^* definisce il percorso statico, il Dynamic Window Approach (DWA) gestisce il controllo in tempo reale. L'algoritmo campiona molteplici traiettorie di velocità (v, ω) nello spazio delle velocità del robot, considerando i vincoli cinematici e dinamici (velocità massima, accelerazione), in pratica in base alla situazione attuale si hanno delle possibili traiettorie da cui scegliere (oppure sotto un aspetto robotico, operazioni da eseguire come per esempio ruotare le ruote o aumentare la vel angolare). La traiettoria ottimale viene selezionata massimizzando la seguente funzione obiettivo G :

$$G(v, \omega) = \sigma[\alpha \cdot \text{head}(v, \omega) + \beta \cdot \text{dist}(v, \omega) + \gamma \cdot \text{vel}(v, \omega)] \quad (5)$$

Le componenti della funzione sono:

- $\text{head}(v, \omega)$: Valuta quanto quella traiettoria sposta il robot nella direzione del target locale (se la traiettoria fa dare le spalle al robot rispetto al target, il punteggio è molto basso).
- $\text{dist}(v, \omega)$: Rappresenta la distanza minima dall'ostacolo più vicino; valori più alti indicano maggiore sicurezza (ostacoli lontani).
- $\text{vel}(v, \omega)$: Misura quanto quella traiettoria fa finire prima il percorso (premia velocità lineari più elevate per garantire l'efficienza operativa).
- σ : Coefficiente di normalizzazione per rendere confrontabili le tre componenti.

I coefficienti di peso (α, β, γ) sono stati calibrati specificamente per l'ambiente del frutteto. Attraverso simulazioni estensive, i parametri sono stati fissati a $\alpha = 0.6$, $\beta = 0.3$, e $\gamma = 0.1$.

Questa configurazione assegna la massima priorità (α) all'orientamento verso l'obiettivo per garantire l'avanzamento, seguita dalla sicurezza rispetto agli ostacoli (β). Il peso della velocità (γ) è mantenuto basso per evitare movimenti aggressivi o instabilità su terreni irregolari, che potrebbero compromettere la qualità delle immagini raccolte durante l'ispezione.

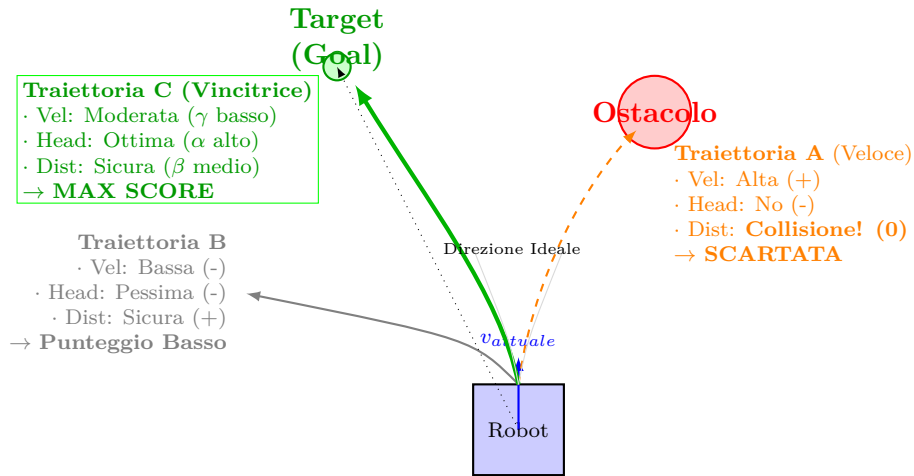


Figura 2: Rappresentazione grafica del Dynamic Window Approach (DWA). Il box verde (Traiettoria C) evidenzia la scelta ottimale che bilancia direzione e sicurezza.

5.3 Metodo Ibrido e Integrazione

L'integrazione dei due algoritmi avviene in modo gerarchico, colmando il divario tra la pianificazione su mappa statica e la navigazione dinamica. Il processo di selezione dei nodi e del target avviene come segue:

Selezione dei Nodi (Livello Globale)

La mappa viene discretizzata in una griglia con risoluzione di **0.5 m** (come definito nella configurazione della Costmap). Ogni cella della griglia rappresenta un nodo candidato.

- L'algoritmo A^* valuta questi nodi e seleziona la sequenza che minimizza la funzione di costo $f(n)$.
- I nodi scelti formano il "percorso globale" (linea rossa nelle visualizzazioni), che è geometricamente ottimale ma non tiene conto della cinematica del robot.

Selezione del Target (Livello Locale)

Il pianificatore locale DWA non punta direttamente alla destinazione finale, ma segue il percorso globale un passo alla volta.

- **Estrazione del Nodo Guida:** Il sistema analizza il percorso globale e seleziona un nodo situato a una certa distanza davanti al robot (lookahead distance). Questo nodo diventa il **Target Locale** temporaneo.
- **Calcolo della Traiettoria:** Il DWA genera il ventaglio di traiettorie possibili e sceglie quella che massimizza la funzione $G(v, \omega)$, puntando verso il Target Locale ed evitando gli ostacoli rilevati in tempo reale dai sensori.
- Man mano che il robot avanza, il Target Locale viene aggiornato scorrendo lungo i nodi del percorso globale, guidando il robot fluidamente fino alla meta.

6 Risultati della Simulazione

Per validare l'approccio proposto, sono state condotte simulazioni in ambiente MATLAB R2022b utilizzando mappe a griglia (grid maps) 2D.

6.1 Confronto Pianificazione Globale

L'algoritmo A^* migliorato è stato confrontato con l'algoritmo A^* standard e l'algoritmo di Dijkstra in termini di tempo di calcolo, lunghezza del percorso e numero di nodi esplorati. I risultati sono riassunti nella Tabella 1.

Tabella 1: Confronto delle prestazioni degli algoritmi di pianificazione globale

Algoritmo	Tempo (s)	Lunghezza Percorso (m)	Nodi Attraversati
A^* Standard	10.5	44.32	427
Dijkstra	18.7	38.79	684
A^* Migliorato	4.3	38.79	374

Come evidenziato dai dati, l' A^* migliorato riduce drasticamente il tempo di pianificazione (da 10.5 s a 4.3 s rispetto allo standard) e il numero di nodi espansi, pur mantenendo una lunghezza del percorso ottimale paragonabile a quella di Dijkstra (38.79 m). Questo dimostra una superiore efficienza di ricerca grazie alla funzione euristica adattiva.

6.2 Confronto Pianificazione Locale

Per la pianificazione locale, il metodo DWA è stato confrontato con l'algoritmo dei Campi di Potenziale Artificiale (APF), un classico metodo reattivo. Le simulazioni hanno mostrato che, sebbene entrambi evitino gli ostacoli, il percorso generato dal DWA risulta significativamente più fluido.

- **Lunghezza traiettoria DWA:** 15.32 m.
- **Stabilità:** A differenza dell'APF, che può soffrire di minimi locali e oscillazioni, il DWA ha mostrato variazioni coerenti e gradualmente della velocità angolare.

Questa "fluidità" (smoothness) si traduce in una postura di movimento stabile e una riduzione del jitter del robot, fattori critici per garantire che le immagini acquisite dalla telecamera durante il movimento non risultino mosse o inutilizzabili.

7 Sperimentazione sul Campo

I test sono stati condotti presso il campus Yangzijin dell'Università di Yangzhou in un ambiente non strutturato.

7.1 Test di Arresto ai Waypoint

Il robot doveva fermarsi per 5 secondi in 4 waypoint designati. La velocità di crociera era impostata a 0.8 m/s.

- **Errore Laterale Medio:** 0.163 m (Max: 0.273 m).
- **Errore Longitudinale Medio:** 0.282 m (Max: 0.526 m).
- **Errore di Orientamento Medio:** 2.9°.
- **Tempi di Reazione:** Frenata media 0.46 s, ripartenza 0.64 s.

Nonostante le deviazioni dovute a irregolarità del terreno, la precisione è stata giudicata sufficiente per le operazioni di ispezione, che richiedono un arresto entro 1 metro.

7.2 Test di Evitamento Ostacoli

È stato valutato l'impatto del parametro `inflation_radius` (raggio di inflazione) sulle performance di evitamento ostacoli (Tabella 2).

Tabella 2: Risultati del test di evitamento ostacoli con vari raggi di inflazione

Raggio (m)	Confusione Traiettoria	Collisione	Dist. Minima (m)	Esito
1.0	Si	No	1.284	Fallito (Disorientato)
0.8	No	No	0.826	Ampio raggio di sterzata
0.6	No	No	0.643	Sterzata brusca
0.4	No	No	0.375	Ottimale
0.3	No	Si	0	Collisione

Il valore ottimale è risultato essere 0.4 m, che bilancia la sicurezza con la capacità di navigare in spazi ristretti senza causare confusione nel pianificatore.

8 Riferimenti Bibliografici per gli Approfondimenti Tecnici

Poiché il paper originale si concentra sull'applicazione specifica e sui risultati sperimentali, le spiegazioni teoriche dettagliate presenti in questo documento (in particolare il funzionamento interno del DWA, la logica delle Costmap in ROS e la discretizzazione spaziale) fanno riferimento alla letteratura standard della robotica mobile. Di seguito sono elencate le fonti primarie per i concetti aggiunti:

Dynamic Window Approach (DWA)

La spiegazione logica del DWA come processo di campionamento delle traiettorie (rappresentato nel diagramma a ventaglio) e la metafora della "votazione" tramite funzione obiettivo fanno riferimento al paper originale che ha introdotto l'algoritmo:

D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance", in *IEEE Robotics & Automation Magazine*, 1997.

[\[IEEE Xplore\]](#) | [\[PDF - CMU\]](#)

Architettura ROS Navigation (move_base)

I concetti specifici di *Rolling Window* (finestra mobile) per la Local Costmap e la gestione tecnica dei layer di inflazione (*Inflation Layer*) sono descritti nel paper che presenta lo stack di navigazione standard di ROS utilizzato in questo studio:

E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, "The Office Marathon: Robust Navigation in an Indoor Office Environment", in *International Conference on Robotics and Automation (ICRA)*, 2010.

[\[IEEE Xplore\]](#) | [\[ROS Wiki Documentation\]](#)

Occupancy Grids e Discretizzazione

La teoria matematica dietro la discretizzazione dello spazio in "nodi" (Occupancy Grid Maps) e la definizione probabilistica di spazio libero/occupato deriva dal testo di riferimento:

S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[\[Sito Ufficiale\]](#)

Riferimenti bibliografici

- [1] Jiwei Qu, Yanqiu Gu, Zhinuo Qiu, Kangquan Guo, Qingzhen Zhu (2025). *Development of an Orchard Inspection Robot: A ROS-Based LiDAR-SLAM System with Hybrid A*-DWA Navigation*. *Sensors*, 25(21), 6662.
<https://doi.org/10.3390/s25216662>